

Usage of Perceptron in prediction of Diabetes with Pima Indians Database

Anonymous CVPR submission

Paper ID 1913414

Abstract

Perceptrons are the most basic neural network architecture. . This paper aims to explore the Perceptron model and its performance in predicting whether a subject has diabetes or not, using data from Pima Indian Diabetes database (available at <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database/data>). We will report on implementing a basic Perceptron model using scikit-learn's Perceptron module and train the model with basic hyperparameters to assess a baseline model before hyperparameter tuning. After trying various hyper-parameter tuning and data resampling techniques, I will implement a Multi Layered Perceptron and a Support Vector Machine with similar settings on the dataset and compare the results of the three models.

1. Introduction of a simple Perceptron

A perceptron is a single layer of a neural network which is trained based on a linear discriminant function:

$$g(X) = WtX + b \quad (1)$$

where Wt is the weight and b is the bias (Murty, MN & Raghava, R 2016, p. 27). The perceptron was invented in the 1950s by Rosenblatt, and its architecture comprised an input and output layer, which the latter is computed as the sum of the dot products of the input values and weights (Jo 2023). The perceptron is initiated with a loss function and tries to find the optimum weight and bias after training data is fed to it and updates the weights continuously through each data point using gradient descent. Gradient descent can be described as trying to find the global minima of a function by starting at a point then proceeding to move towards the direction of the steepest descent until we reach the global minima (Wegner, SA 2024)

Once the optimum weights and bias are set after training, the perceptron model can be used for predictions. The model makes a prediction based on the weighted sum through an activation function and outputs an appropriate

value. In its simplest implementation, an activation function can be a step function or a sigmoid function, where the model predicts a class of 0 or 1, based on whether the weighted sum is above or below a threshold value.

Because of how it is implemented, Perceptrons work because of the principle of linear separability; dimensions of the two classes must be linearly separable in order for the perceptron to effectively create a decision boundary. A perceptron is suitable for classification tasks where the difference between the classes are linearly separable, and in this paper a perceptron machine learning model will be trained and fine tuned to predict whether a person has diabetes using hyperparameter tuning and data resampling techniques.

1.1. Hyperparameters

Hyperparameters are settings of a machine learning model which can be tuned in order to change how the model is trained. Tuning hyperparameters can help a model be trained optimally. Below are the hyperparameters that I will be tuning in this experiment.

1. Learning Rate - the rate at which the weights is updated. I will be using a range from $1e-3$, down to $1e-5$ in this paper, as a learning rate that is too fast will overshoot the minimum while a learning rate that is too slow may prevent convergence.

2. Tolerance - the threshold that the weight needs to be updated. If the weight update does not exceed the tolerance over a number of runs, the iterations will stop to save time and prevent overfitting.

3. Regularization - The type of regularization method used to prevent over-fitting of the model to the training set. I will try out L1, L2, ElasticNet and no regularization.

4. Regularization Strength - the strength or level at which regularisation is implemented. I will experiment with different rates (0.001 to 0.1) to prevent over-fitting while maintaining model performance.

5. Max Iterations - Number of times the model can perform a full epoch over the training set during training. I use the range 100, 500, 1000 and 2000. A low number may prevent convergence but numbers too high will increase training time. This will be balanced with early stopping in order

to limit the iterations once there isn't any significant improvement in weights.

6. Early Stopping - The early stop function stops the training if the weight update doesn't exceed the tolerance value set. This will always be set to True in this experiment so that the model stops iterating through the training process if no improvements are made.

7. Class weight - Class weight will be set to 'balanced' to try to maintain a balance between Diabetic (1) and Non-diabetic (0) classes.

1.2. Methodology

I will import scikit-learn's Perceptron module and initiate it with default settings which will serve as my baseline model and train it on the dataset to get a baseline performance. I then use GridSearchCV to fine tune the hyperparameters of the model in order to optimize the model's performance.

1.3. Dataset

For any machine learning model, data cleaning is required and is also considered best practice. The quality of the data that a model is trained on also directly correlates with the quality of the model itself. *Garbage in, garbage out*, as the common saying goes.

In the Pima Indians Database, there are 768 rows of data with 9 columns, 8 of which are features and 1 'outcome' column which is the label. After loading the dataset and inspecting it, I found that data cleaning is required in order to train the model effectively:

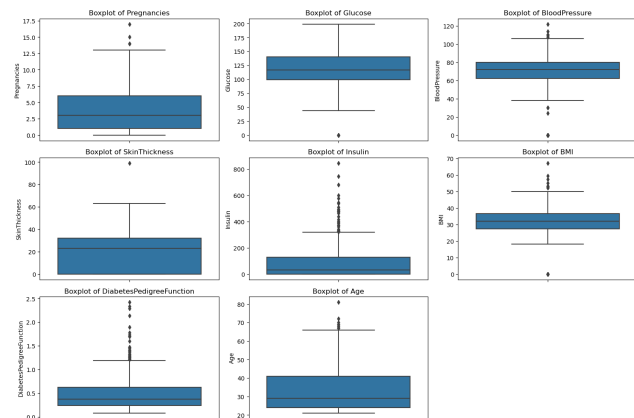


Figure 1. Uncleaned Dataset

From Figure 1, data cleaning required:

1. Missing values in 'Glucose', 'Blood Pressure', 'Skin Thickness', 'Insulin', 'BMI' which are input as 0 which are erroneous. Due to the small size of the dataset, I impute missing values with K-Nearest Neighbours (set at 5) to try and get a reasonable value instead of just using the median/mean.

2. The small size of the dataset also discourages removing any rows of data, even outliers. Outliers will instead be capped at upper and lower bounds using InterQuartile Range (IQR) to prevent them from skewing the data too much.

3. 'Glucose', 'Blood Pressure', and 'BMI' are scaled using StandardScaler

4. 'Pregnancies', 'SkinThickness', 'Insulin', 'DiabetesPedigreeFunction', 'Age' are scaled using MinMaxScaler.

After cleaning the dataset, the distribution in Figure 3 looks much better and should contribute to overall better training for the model:

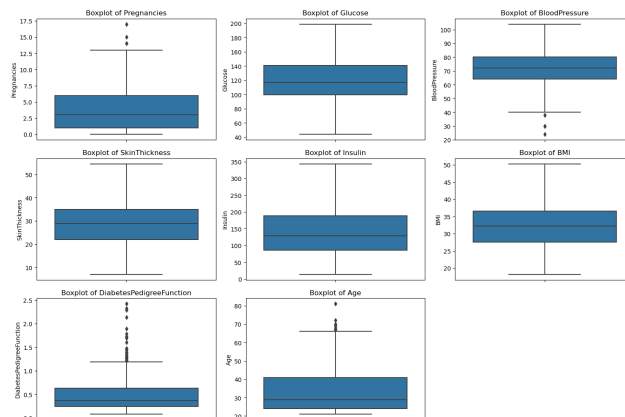


Figure 2. Cleaned Dataset

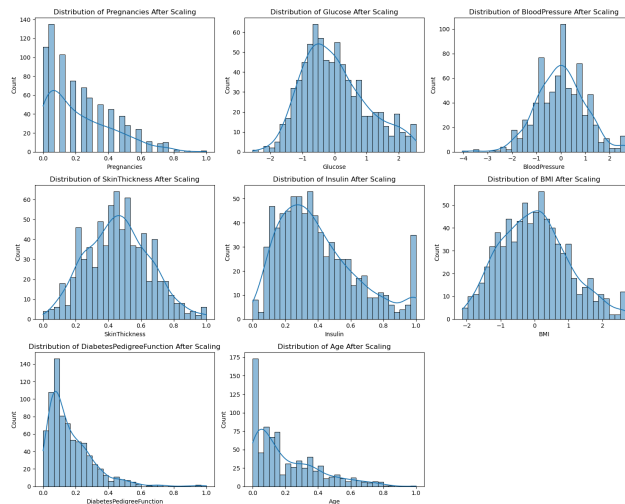


Figure 3. Distribution of Scaled and Cleaned Data

Lastly, before I begin to define and train my model, I use a correlation matrix to check for any features that I might have to remove:

Figure 4 shows no highly correlated features, so there doesn't seem to be any redundant features to perform di-

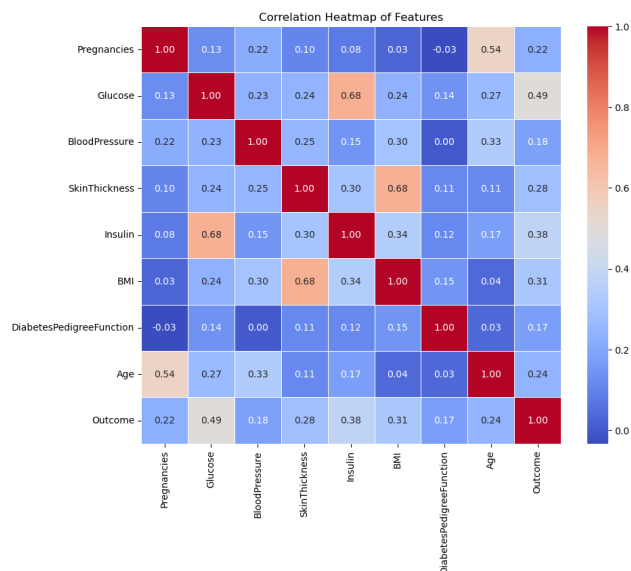


Figure 4. Correlation Matrix of Features

mension reduction on. It is also noted that 'Glucose', 'Insulin' and 'BMI' has the highest correlation with 'Outcome', which is logical. Overall, Figure 3 and Figure 4 is showing strong signs that the dataset is cleaned properly and is now ready for use in training.

2. Implementation, Testing and Comparison of Perceptron Model

2.1. Initializing the Baseline Perceptron model

The baseline perceptron model will be initiated using scikit-learn's Perceptron module. This model will serve to obtain a reasonable baseline performance of a perceptron before hyperparameter tuning is performed. By default, scikit-learn's Perceptron uses Stochastic Gradient Descent as the weight update rule, and the activation function is Heaviside step function. I initiate the perceptron with a setting of maximum iteration of 1000 and tolerance of 0.01 and begin training on the cleaned dataset. The performance will be logged in Table 1.

Table 1. Baseline Perceptron

Metric	Precision	Recall	F1-Score
Class 0	0.76	0.68	0.72
Class 1	0.51	0.61	0.55
Macro Avg	0.6		

2.2. Hyperparameter Tuning

I used GridSearchCV to perform hyperparameter tuning on the baseline model. GridSearchCV iterates through the

parameter grid and cross-validates it before returning the settings with the best scoring metric, which was set as 'accuracy'.

The parameter grid:

1. Learning Rate: [0.001, 0.01, 0.1, 1.0]
2. Tolerance: [0.001, 0.0001, 0.00001]
3. Maximum Iterations: [100, 500, 1000, 2000]
4. Regularization / Penalty: [None, 'l2', 'l1', 'elastic-net']
5. Regularization Strength: [0.0001, 0.001, 0.01, 0.1]

The best settings for the model returned by GridSearchCV was:

Regularization Strength: 0.001

Learning Rate: 0.1,

Maximum Iterations: 100,

Penalty: L1

Tolerance: 0.001.

This setting will be the hypertuned perceptron. I then initiated the Perceptron module once again with these settings applied and present the performance in Table 2.

Table 2. Hypertuned Perceptron

Metric	Precision	Recall	F1-Score
Class 0	0.75	0.84	0.79
Class 1	0.62	0.48	0.54
Macro Avg	0.69	0.66	0.71

The poor performance in Recall and F1-Score for Class 1 (Diabetic) subjects might be due to class imbalance. I add class balance to check if performance improves.

Table 3. Hypertuned & Class Weighted Perceptron

Metric	Precision	Recall	F1-Score
Class 0	0.90	0.65	0.76
Class 1	0.57	0.87	0.69
Macro Avg	0.73		

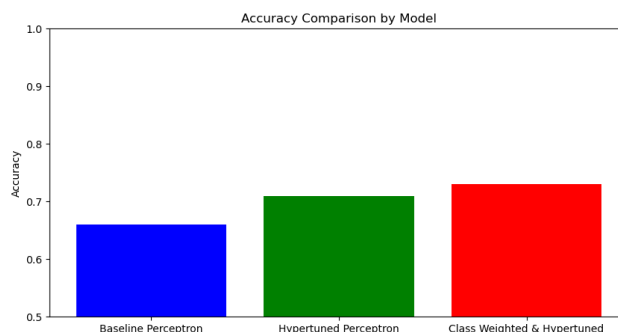


Figure 5. Accuracy across models

Even after setting tuning hyperparameters and setting weights for class imbalance, it seems that performance does not increase significantly. Baseline Perceptron has a balanced but moderate performance, with lower recall for class 1 (diabetics). Hypertuned Perceptron improves accuracy and class 0 metrics, but struggles slightly with recall for class 1. Class Weighted & Hypertuned significantly increases recall for class 1, indicating better attention to the minority class at the cost of some loss in precision.

This leads me to think that perhaps the dataset itself might not have enough to train a perceptron to predict the classes effectively or the difference between the two classes might not be linearly separable.

2.3. Alternative Models - Multi Layer Perceptron & Support Vector Machine

Further experimentations with Multi Layer Perceptrons also did not yield much better performance. Even after hyperparameter tuning and trying out different depths and layers of MLP, the performance was more or less the same.

Table 4. Multi Layer Perceptron (highest accuracy)

Metric	Precision	Recall	F1-Score
Class 0	0.79	0.81	0.80
Class 1	0.63	0.61	0.62
Macro Avg			0.74

A support vector machine is a classifier that is able to find the non-linear decision boundaries. It operates by the principle of finding the boundaries in hyperplane which splits the different classes (Nazir, T, Qi, X & Silvestrov, S 2017). It also performs well in the case of the dataset which is small. I implemented a Support Vector Machine on this dataset to see if there is a significant improvement in prediction performance and log the results in table 4.

Table 5. Support Vector Machine

Metric	Precision	Recall	F1-Score
Class 0	0.75	0.86	0.80
Class 1	0.65	0.48	0.55
Macro Avg			0.73

3. Conclusion

In conclusion, after performing data cleaning to the best of my abilities and trying out all the different techniques that I could, the Perceptron model was only able to reach up to 0.75 accuracy. This also applied to each of the other alternative machine learning models that I tried to train with the given dataset, including a Multi Layered Perceptron and a Support Vector Machine. Such a level of accuracy in the

context of medical field is generally unacceptable and therefore the models discussed here if only trained on the given dataset should not be deployed for now.

The small sample size of 768 rows and large proportion of missing values that needed to be imputed are likely to have inherent limitations which led to an upper limit for what a machine learning model can achieve.

Moving forward, I am confident that any of the three models implemented in this experiment can be raised to satisfactory levels of accuracy given that the quality and comprehensiveness on which the models are trained are of the highest quality.

References

- Jo, T 2023, *Deep Learning Foundations*, 1st ed. 2023., Springer International Publishing, Cham.
- Murty, MN & Raghava, R 2016, *Support Vector Machines and Perceptrons: Learning, Optimization, Classification, and Application to Social Networks*, 1st edn, Springer International Publishing AG, Cham.
- Nazir, T, Qi, X & Silvestrov, S 2017, 'Linear Classification of Data with Support Vector Machines and Generalized Support Vector Machines', in *Engineering Mathematics II*, vol. 179, Springer International Publishing AG, Switzerland, pp. 355–375.
- Wegner, SA 2024, *Mathematical Introduction to Data Science*, 1st ed. 2024., Springer Berlin Heidelberg, Berlin, Heidelberg.